

FORENSIC REPORT · KSR-LQD-001

The fix created the flaw that was used

A rangeproof cache-key collision in Elements, introduced by [c26d719c29](#), and used to admit an output carrying an invalid proof into Liquid block 4050336.

TARGET

Liquid Network · Elements Core

INCIDENT

6 September 2026

BLOCK

4050336

SEVERITY

CRITICAL

ANALYSIS DATE

7 September 2026

VERSION

1.0

AUTHOR

Kairos Lab by Valisthea

STATUS

Mechanism reproduced

DISTRIBUTION

Public

01 Executive summary

Verdict

Transaction f24a...183f carries an output whose rangeproof is cryptographically invalid. It was accepted anyway, because the node that validated it never ran that check. It found a "valid" result already sitting in its cache, deposited one block earlier by an innocuous-looking transaction.

The two cache entries are identical only under the patched keying. Under the published vulnerable build they are distinct and the attack fails. Commit c26d719c29, written to close one cache defect, opened a second and more exploitable one, and that is the one that was used.

In plain terms

A Liquid node remembers which proofs it has already checked, so it does not repeat expensive work. Each proof is filed under a label. The label is built by gluing four pieces of information together and hashing the result.

The glue carries no marker for where one piece ends and the next begins. Two completely different outputs can therefore produce exactly the same byte string, and therefore the same label. The attacker built a pair of outputs in precisely that situation: the first is legitimate and passes verification normally, which files the label; the second is fraudulent, claims the same label, and is handed the first one's result without any verification taking place.

The fraudulent output acts as an accounting counterweight. It lets the transaction look balanced while it is in fact creating value. That value later leaves the chain through two peg-outs totalling **3,998.66973280 BTC**.

Key results

FINDING	STATUS	SECTION
The attack "rangeproof" is the primer's cache preimage, not a proof	PROVEN	§4.3
Both post-fix key preimages are byte-identical (4,301 B)	PROVEN	§4.4
No collision under the pre-fix key (4,199 B vs 4,267 B)	PROVEN	§4.4
The collision is independent of the per-process salt (2,000 / 2,000)	MEASURED	§5.2
Primer verifies TRUE, attack verifies FALSE, in native secp256k1-zkp	MEASURED	§5.1
A patched, primed node accepts 4050336; a cold or unpatched node rejects it	MEASURED	§5.3
The accepting nodes ran the patched build, which was never in a release	INFERRED	§6.1
Identity of the first acceptor, node logs, version inventory	UNVERIFIED	§8

Primary recommendation

Any cache key built by concatenating variable-length fields must frame each field with its length, or use per-field tagged hashing. The fix proposed in §7 has been implemented and tested: it removes the collision and makes the primed node reject f24a...183f.

02 Scope and method

Objects analysed

ROLE	IDENTIFIER	BLOCK	OUTPUT
Attack	f24a4b179b5cc7e88b25a763911f7cbdf2bf45d1d1b5ab611e94461cef0a183f	4050336	vout 1
Primer A	71c93d4339fe8328981a2ec0dd23808d1ff104dc4c4cbcf5c06fb2bb622f411	4050335	vout 0
Primer B	271147100a94f6337b6c3db39b30c92d5b97ed91597307b6f721f73a15187ec5	4050335	vout 0
Block	e1d9a2aae69e0fc3ca18f7f7f84e0615e92a5e3b5000d66c10c34043346da0d5	4050336	7 tx, f24a at index 1

The two primers' vout 0 are **byte-identical** — same proof, same commitment, same script; the proof hashes to `a3c000d3...b45ffb` in both. The duplication is propagation redundancy, not two distinct primers.

Reference code

ITEM	VALUE
Commit analysed	c26d719c29a40da280a825b25657e9c3d8bc7d99
Author / dates	Byron Hambly <bhambly@blockstream.com> · authored 2026-08-03 · committed 2026-09-01
Merged	PR #1592 → 31e8f27f4b (2026-09-01)
Backports	6253d7e103 (PR #1595, 23.x, 2026-09-02) · 212c43f475 (PR #1599, 23.3.x, 2026-09-03)
Last release before the incident	elements-23.3.3 (2026-04-13), pre-fix keying
Upstream files read	src/script/sigcache.cpp, src/confidential_validation.cpp, src/cuckoocache.h (master)

Reproduction toolchain

COMPONENT	DETAIL
Cryptography	ElementsProject/secp256k1-zkp 037cc6d74cbb4a89e443117459b577d56a582e54 (2026-09-05), modules generator + rangeproof + surjectionproof
Cache	src/cuckoocache.h upstream, unmodified
Compiler	gcc / g++ 16.1.0 (MinGW-w64), -O2 -std=c++17
SHA-256	Streaming reimplementation, API-equivalent to CSHA256, self-tested against a NIST vector on every run
Data	Raw hex extracted from chain; digests in Appendix A

What was not done

- No full `elements` was built or run. The `ConnectBlock` path is reproduced at the rangeproof checker and its cache, not at the chainstate.
- No production node logs were obtained. The identity of the first acceptor remains unknown.
- No numerical opening of commitment C1 was attempted; it requires the blinding factor.

03 The question left open

The prior investigation established two solid facts and a gap between them.

ESTABLISHED	
A	The rangeproof of f24a...183f:1 is invalid under its real context (L-BTC, script 6a). Native verification: FALSE.
B	Block 4050336 is federation-signed and sits in the best chain. Its descendants spend f24a's outputs.

The gap: under stock Elements with an empty cache, ConnectBlock must reject f24a at the vout 1 rangeproof. How was this transaction actually accepted?

The natural lead, cache poisoning, had been ruled out: it requires a prior successful verification of the *same* (proof, commitment) pair, and that pair exists nowhere else on chain. One hundred and twenty-eight candidate contexts were tested, zero hits.

That conclusion was correct, and the inference drawn from it was wrong. The search targeted the *pre-fix* key, which retains only the proof and the value commitment. Under that key the primer transactions genuinely cannot deposit the needed entry. Under the *post-fix* key they can, and that is precisely what they were manufactured to do.

Restating it

The right question is not "which context makes (P1, C1) valid". No such context exists, and none is needed. The right question is: **which other output produces the same cache label** as f24a...183f:1. The answer is in the preceding block.

04 Mechanism

4.1 The original defect and its fix

Before c26d719c29, a rangeproof's label retained two fields while verification consumes four. Two contexts differing only in asset generator or script therefore shared one entry.

```
// before - src/script/sigcache.cpp
hasher.Write(proof.data(), proof.size())
        .Write(commitment.data(), commitment.size())
        .Finalize(entry.begin());

// after - c26d719c29, two fields added, no framing
hasher.Write(proof.data(), proof.size())
        .Write(commitment.data(), commitment.size())
        .Write(asset_commitment.data(), asset_commitment.size())
        .Write(scriptPubKey.data(), scriptPubKey.size())
        .Finalize(entry.begin());
```

Before the fix the key held two fields, one of them fixed at 33 bytes, so the split was unique and the encoding injective. After the fix it holds **four fields, two of them variable-length**, concatenated with no length prefix and no domain separator. The encoding stops being injective: several distinct quadruples produce the same byte string, hence the same label.

4.2 Which bytes actually enter the key

This is the pivot. The `asset_commitment` handed to the cache is not the wire asset field. For an explicit asset, Elements replaces it with the **serialized generator** before calling the checker.

```
// src/confidential_validation.cpp:379-383 (master)
if (asset.IsExplicit()) {
    int ret = secp256k1_generator_generate(ctx, &gen, asset.GetAsset().begin());
    assert(ret != 0);
    secp256k1_generator_serialize(ctx, &vchAssetCommitment[0], &gen);
}
QueueCheck(checks, new CRangeCheck(&val, ptxoutwit->vchRangeproof,
                                   vchAssetCommitment, tx.vout[i].scriptPubKey, store_result));
```

This is structurally forced: `VerifyRangeProof` then calls `secp256k1_generator_parse`, which accepts only the `0x0a` and `0x0b` prefixes. An explicit asset in wire form (`01 || assetid`) would not parse.

FIELD	VALUE FOR F24A:1 AND FOR BOTH PRIMERS
Wire asset	016d521c38ec1ea15734ae22b7c46064412829c0d0579f0a713d1c04ede979026f
Entering the key (generator)	0a0a488de4899d0ae757f6cf8368663184d164106111ed9eaecf510e35282ddc6d

All three outputs carry the same explicit L-BTC asset, so the derived generator is identical across them. That is the entry condition for the collision. Derivation re-run here in native `secp256k1-zkp`, §5.1.

4.3 What the two objects really are

The attack's 4,234-byte "rangeproof" is not a proof. It is the primer's cache preimage, extended by 35 bytes:

```
P1 = P0 || C0 || G || 0x6a || 0x43      4166 + 33 + 33 + 1 + 1 = 4234 bytes
```

And the primer's 69-byte script is not an arbitrary data carrier. It is the tail of the attack's preimage:

```
S0 = 0x6a || 0x43 || C1 || G || 0x6a    OP_RETURN PUSH67 <C1 || G || 0x6a> = 69 bytes
```

The two objects are cut for each other. Each carries exactly the bytes the other is missing to complete the same 4,301-byte string.

4.4 The alignment

Both outputs hand the hasher exactly the same 4,301 bytes, split at different field boundaries.

■ read by the attack ■ read by the primer

OFFSET	CONTENT	F24A...183F:1 READS IT AS	71C93D43:0 READS IT AS
[0 : 4166]	P0 – primer's proof	proof	proof
[4166 : 4199]	C0 – primer's value commitment	proof	value_commitment
[4199 : 4232]	G – L-BTC generator	proof	asset_commitment
[4232 : 4234]	6a 43 – OP_RETURN PUSH67	proof	scriptPubKey
[4234 : 4267]	C1 – attack's value commitment	value_commitment	scriptPubKey
[4267 : 4300]	G – L-BTC generator	asset_commitment	scriptPubKey
[4300 : 4301]	6a – OP_RETURN	scriptPubKey	scriptPubKey

The hinge is at offset 4,234. To its left the attack is still inside its proof field while the primer has already crossed into its script. The two meet on the final byte. SHA-256 sees only the string, never the split.

KEY	PRIMER	ATTACK	RESULT
Pre-fix	4,199 B	4,267 B	different lengths, no collision
Post-fix	4,301 B	4,301 B	identical preimages, same cache entry

05 Reproduction

5.1 Cryptographic ground truth

Generator derivation and native verification of both contexts, upstream secp256k1-zkp.

```
== 1. vchAssetCommitment as Elements builds it (explicit asset -> serialized generator)
f24a:1      wire asset      len=33 prefix=0x01
71c93d43:0  wire asset      len=33 prefix=0x01   same asset id as f24a:1 : YES
derived generator f24a:1      0a0a488de4899d0ae757f6cf8368663184d164106111ed9eaecf510e35282ddc6d
derived generator 71c93d43:0  0a0a488de4899d0ae757f6cf8368663184d164106111ed9eaecf510e35282ddc6d
identical for both outputs:    YES
G found inside attack proof P1: YES (offset 4199)
G found inside primer script S0: YES (offset 35)

== 2. structural identity of the attack rangeproof
|P1|=4234 |P0|=4166 |C0|=33
P1 == P0 || C0 || G || 0x6a || 0x43 : TRUE
S0 == 0x6a || 0x43 || C1 || G || 0x6a : TRUE

== 3. native secp256k1_rangeproof_verify on both contexts
primer 71c93d43:0 (C0,P0,script=69B,G) -> TRUE <-- stores cache entry   min=0 max=4503599627370495
attack f24a:1     (C1,P1,script=1B ,G) -> FALSE <-- never reached on a cache hit

VERDICT: structural=1 script=1 primer_true=1 attack_false=1
```

The primer verifies true with `min_value = 0`, which is permitted because its script is unspendable (OP_RETURN). It therefore clears the policy check and the entry is stored.

5.2 Cache keys

Faithful replica of `SignatureCache`, with the 32-byte salt written twice as upstream does, over unmodified `cuckoocache.h`.

```
sha256 self-test = ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
                  expected ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad

=== PRE-FIX key : H(salt || proof || value_commitment)
primer preimage 4199 B key f462ee33ca4c7d56ba336ae04dc45c8d26e9113681db989ec5425871f8c9b9bb
attack preimage 4267 B key 8a11f3f7ca49bc03623a662de2db15b0f263cf2086ea08a9eb70e81c6c0804cb
SAME CACHE ENTRY : no

=== POST-FIX key : H(salt || proof || value_commitment || asset_commitment || scriptPubKey)
primer preimage 4301 B key 8026a5bf5c2f55c9132bbe09b6f75dfa05a4509086b1b6bde0996481f04683d4
attack preimage 4301 B key 8026a5bf5c2f55c9132bbe09b6f75dfa05a4509086b1b6bde0996481f04683d4
SAME CACHE ENTRY : YES

=== FRAMED key : H(salt || len:proof || len:value_commitment || len:asset_commitment || len:scriptPubKey)
primer preimage 4317 B key b266607b0c5b74b2090d74c17c067cb54746a94afc19957860478b37ec5dde4b
attack preimage 4317 B key f87c8b8656d0e4b2649b3565bdbd99341a137aa0edc48470276969ffefe7cb39
SAME CACHE ENTRY : no

post-fix collision over 2000 independent per-process salts : 2000/2000
```

The salt is drawn at random on every process start. The collision does not depend on it: this is a preimage identity, not a hash accident. Two thousand independent salts, two thousand collisions.

5.3 Validation-path replay

Replica of `CachingRangeProofChecker::VerifyRangeProof` with real cryptography underneath, across three node profiles plus the proposed fix.

```

---- node A post-fix build, up since < 4050335, primer transited its mempool
ATMP 4050335 primer store=true -> TRUE crypto TRUE, entry stored
ATMP 4050336 attack store=true -> TRUE CACHE HIT (crypto skipped)
CONNECT 4050335 primer store=false -> TRUE CACHE HIT (crypto skipped)
CONNECT 4050336 attack store=false -> TRUE CACHE HIT (crypto skipped)
=> block 4050336 : ACCEPTED

---- node B post-fix build, cold start / IBD, primer only seen inside a block
CONNECT 4050335 primer store=false -> TRUE crypto TRUE
CONNECT 4050336 attack store=false -> FALSE crypto FALSE
=> block 4050336 : REJECTED

---- node C pre-fix build (elements-23.3.3), primer transited its mempool
ATMP 4050335 primer store=true -> TRUE crypto TRUE, entry stored
ATMP 4050336 attack store=true -> FALSE crypto FALSE
CONNECT 4050335 primer store=false -> TRUE CACHE HIT (crypto skipped)
CONNECT 4050336 attack store=false -> FALSE crypto FALSE
=> block 4050336 : REJECTED

---- node D proposed length-framed key, primer transited its mempool
ATMP 4050335 primer store=true -> TRUE crypto TRUE, entry stored
ATMP 4050336 attack store=true -> FALSE crypto FALSE
CONNECT 4050335 primer store=false -> TRUE CACHE HIT (crypto skipped)
CONNECT 4050336 attack store=false -> FALSE crypto FALSE
=> block 4050336 : REJECTED

```

This is the exact shape of the observed disagreement. One block, one byte string, two opposite verdicts, decided by the process's memory history. No consensus-version divergence, no missing signature: only a cache.

5.4 Why the entry survives to block connection

The entry is only ever inserted by the mempool. `ConnectBlock` calls `Get(entry, erase = true)` and never inserts. One might expect connecting block 4050335 to consume the entry before 4050336 needs it. It does not:

```

// src/cuckoocache.h - upstream
inline bool contains(const Element& e, const bool erase) const {
    std::array<uint32_t, 8> locs = compute_hashes(e);
    for (const uint32_t loc : locs)
        if (table[loc] == e) {
            if (erase) allow_erase(loc); // bit_set; the element stays in the table
            return true;
        }
    return false;
}

```

`allow_erase` only marks the slot as reclaimable. The value stays comparable until a later insertion overwrites it. Node A above shows this empirically: connecting 4050335 touches the entry with `erase = true`, and connecting 4050336 still finds it.

06 Reading the incident

6.1 The inversion

The public narrative attributes the incident to an unpatched vulnerable build. The mechanism says the opposite.

NODE PROFILE	KEY	PRIMER SEEN IN MEMPOOL	BLOCK 4050336
Federation / continuously running node	post-fix	yes	accepted
Patched node restarted or resyncing	post-fix	no	rejected
Public node on 23.3.3	pre-fix	either	rejected

An attacker who believed the network was running the released build would have constructed a *pre-fix* collision, which is trivial: reuse the same proof and commitment under a different generator. He did the exact opposite. He manufactured a 4,301-byte preimage that collides only under the patched key, which was published on 1 September and backported to the 23.3.x branch on 3 September. The attack lands on the 6th.

Consequence: the attacker knew his target was running the patched build, which was never shipped in a release. The disclosure window became the attack window, and the patch published in the clear served as the specification for the exploit.

Status: **INFERRED** — intent is inferred from the structure of the objects, whose chance of arising accidentally is nil in practice. The actual profile of the accepting nodes still has to be established from logs (§8).

6.2 On-chain effect

Output vout 1 is an OP_RETURN holding a confidential value. It is never spent, but its commitment enters the transaction's Pedersen tally. It is the counterweight.

```
TALLY full (in0 vs out0+out1+out2+fee58)    => 1
TALLY without out1 (in0 vs out0+out2+fee58) => 0
```

The transaction balances only with C1 present. That is exactly why the rangeproof had to be bypassed: a real proof would establish $0 \leq v < 2^{52}$ and forbid the value that makes the tally hold. The created value leaves through the spendable outputs.

HEIGHT	TRANSACTION	EFFECT
4050333	0fbde521...	creates the input f24a spends
4050335	71c93d43... / 27114710...	deposit the cache entry
4050336	f24a...183f	invalid-proof output accepted
4050337	3875a6d6...	spends f24a:0
4050343	c6ea588a...	spends f24a:2
4050344	46f117...	peg-out 2.65138358 BTC
4050349	ce4cae...	peg-out 3,996.01834922 BTC
BTC 965783	8db751a650...b140	settles both amounts — 3,998.66973280 BTC

07 Remediation

7.1 Immediate fix

Frame each field with its length. The hash becomes injective again and the split becomes unique.

```
// src/script/sigcache.cpp
static void WriteLen(CSHA256& h, size_t n) {
    unsigned char le[4] = {(unsigned char)(n),      (unsigned char)(n >> 8),
                          (unsigned char)(n >> 16), (unsigned char)(n >> 24)};

    h.Write(le, 4);
}

void SignatureCache::ComputeEntryRangeProof(uint256& entry,
    const std::vector<unsigned char>& proof,
    const std::vector<unsigned char>& commitment,
    const std::vector<unsigned char>& asset_commitment,
    const CScript& scriptPubKey) const {
    CSHA256 hasher = m_salted_hasher_range_proof;
    WriteLen(hasher, proof.size());      hasher.Write(proof.data(), proof.size());
    WriteLen(hasher, commitment.size());  hasher.Write(commitment.data(), commitment.size());
    WriteLen(hasher, asset_commitment.size()); hasher.Write(asset_commitment.data(), asset_commitment.size());
    WriteLen(hasher, scriptPubKey.size()); hasher.Write(scriptPubKey.data(), scriptPubKey.size());
    hasher.Finalize(entry.begin());
}
```

Tested. Under this key both preimages are 4,317 bytes and hash to b266607b... versus f87c8b86.... Node D in §5.3, primed identically, rejects f24a...183f.

7.2 Reach of the same defect

The pattern is a class defect, not a local accident. Any label built from unframed concatenation of variable-length fields is suspect. To audit in the same repository:

```
ComputeEntrySurjectionProof(entry, hash, proof, commitment)
hash      32 bytes, fixed      ok
proof     variable length      unframed
commitment 33 bytes, fixed      ok
-> only one variable field today, so no split ambiguity yet; the moment a
    second variable field is appended, this key becomes the rangeproof key.
```

Not exploited here. Flagged as a surface to gate before any field is added to that function, since adding fields is exactly what turned the rangeproof key into an attack surface.

7.3 Structural hardening

- **Tagged hashing.** Prefix a constant domain tag per key function, BIP-340 style, rather than relying on field order.
- **Test invariant.** Add a non-injectivity property to the test suite: for any two distinct contexts, the keys must differ. A fuzzer over field lengths would have found this collision in seconds.
- **Disclosure policy.** A consensus fix pushed in the clear to a public repository ahead of deployment exposes the target for the whole window. Here the fix did not merely reveal the original defect, it published the manual for the second one.
- **Targeted review.** Any change to a cache key function should be treated as a consensus change and reviewed accordingly.

08 What remains to be established

The mechanism is reproduced. Applying it to the real nodes of the incident rests on two links that only operational logs can close.

#	LINK	STATUS	HOW TO CLOSE IT
1	The nodes that connected 4050336 were running a post-fix key	INFERRED	Version inventory of the functionaries on 2026-09-06
2	The primer transited those nodes' mempools	INFERRED	debug.log mempool acceptance at 4050335
3	Identity of the first acceptor	UNKNOWN	ProcessNewBlock / ActivateBestChain logs
4	Numerical value committed by C1	UNKNOWN	Requires the blinding factor; not publicly recoverable
5	Competing block at 4050336 on a rejecting branch	UNKNOWN	Tip dump from a node that stalled

What would falsify this report

- A post-fix elementsd run, primed by 4050335, that **still rejects** 4050336.
- Evidence that the functionaries were running a pre-fix key on 6 September. The collision would remain real, but would not be the acceptance path that was used.
- A difference between the SignatureCache replica used here and the real Elements CSHA256. The NIST self-test on every run makes this unlikely, but it is not excluded until a full binary is built.

What none of these links affect: the existence of the collision. It is a property of published code and two confirmed transactions. It is exploitable today against any node running c26d719c29, 6253d7e103 or 212c43f475 without the framing of §7.1, independently of what actually happened on 6 September.

Classification

Severity	CRITICAL — confidential-amount verification bypass on the consensus path
CVSS 4.0 vector	CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H
Numeric score	not independently computed; the band is assigned on impact
CWE	CWE-694, duplicate identifier for two distinct resources. Root pattern: non-injective encoding into a hash preimage
Attack requirement	AT:P — the primer must transit the target's mempool and the target must run the post-fix key
Status	Open at time of analysis

A Appendix — evidence and replay

A.1 Input digests

FILE	SHA - 256
out1_rangeproof.hex	4eb04256615c703f73552c225627602312b1d168e1fb1681ddfc60291e06d3ea
out1_commitment.hex	77641164b22bdf9fe6224b5a04b65d821a7d6021cc0806623d6e71024ee61951
out1_asset.hex	fdb98f8f5792f7eada7b2bd3fde879aeb23a1fa78232ef0fdfd7773de96f5be9
out1_script.hex	0e9deef32abfc454392d21725f9defeff1f79a1528f24373d20877bacd697959
71c93d43_out0_rangeproof.hex	a3c000d30c8b07203610861a28bde1195204980e766a4cdf19a6b5db97b45ffb
71c93d43_out0_commitment.hex	ffe0660510cc16c0e7a649dd7b74c0524785d84dae90f245083b9cb6157d2f78
71c93d43_out0_script.hex	fd16e516563e5a312bf3372e3711fe4f5d45f30078566ad2969cd362fa7d868a
27114710_out0_rangeproof.hex	a3c000d30c8b07203610861a28bde1195204980e766a4cdf19a6b5db97b45ffb

The last two rows share a digest: the primers are twins.

A.2 Reference values

P1 — SHA-256	6619fa29ce0967ae93baefef0bc6233972b05249a9861a96695a106bbcc098b2
C1	086f5d67160fc4b477954fb09ef321e5b589d7a07740a1a6df494ed2335b1d01d8
CO	09d6c6150e95e576d288dcc890bf48518119c6f227ea1be6f9a932792dbec683f5
G — L-BTC generator	0a0a488de4899d0ae757f6cf8368663184d164106111ed9eaecf510e35282ddc6d
Shared post-fix key	8026a5bf5c2f55c9132bbe09b6f75dfa05a4509086b1b6bde0996481f04683d4

The shared key above corresponds to the harness's deterministic salt. In production the salt is random per process, and the collision holds regardless: see the 2,000-salt test in §5.2.

A.3 Replay

```
git clone --depth 1 https://github.com/ElementsProject/secp256k1-zkp.git zkp

gcc -O2 -DSECP256K1_STATIC -Izkp -Izkp/include -Izkp/src \
  -DENABLE_MODULE_GENERATOR=1 -DENABLE_MODULE_RANGEPROOF=1 \
  -DENABLE_MODULE_SURJECTIONPROOF=1 -DECMULT_WINDOW_SIZE=15 -DECMULT_GEN_PREC_BITS=4 \
  -c zkp/src/secp256k1.c zkp/src/precomputed_ecmult.c zkp/src/precomputed_ecmult_gen.c
ar rcs libsecp.a secp256k1.o precomputed_ecmult.o precomputed_ecmult_gen.o

# ground truth: object structure + native crypto verdicts
gcc -O2 -DSECP256K1_STATIC -Izkp/include gt_crypto.c libsecp.a -o gt_crypto
./gt_crypto

# key collision + validation-path replay over upstream cuckoocache.h
g++ -std=c++17 -O2 -DSECP256K1_STATIC -I. -Ishim -Izkp/include cache_split.cpp libsecp.a -o cache_split
./cache_split
```

gt_crypto exits 0 if and only if all four assertions hold: structure of P1, structure of S0, primer true, attack false.

A.4 Disclaimer

This document reports an independent reconstruction carried out from public chain data and public source code. It does not purport to identify any person or to establish intent in a legal sense. Items marked "Inferred" are inferences, stated as such; items marked "Unverified" have not been established. Any operational decision taken on the basis of this report should await closure of the links in §8.